

接入文档-Android

喜马拉雅生态SDK Android集成指南

概述

喜马拉雅生态SDK (XMEcoSDK) 为第三方应用提供音频内容服务，主要能力包括：音频播放控制、WebView 内容接入、用户认证、悬浮播放器、锁屏播控、数据获取等。

系统要求

- 最低版本：Android 7.0+ (API Level 24+)
- Android Gradle Plugin：8.1.1+ (compileSdk 34 要求)
- Kotlin：1.9+ / Java：JDK 17

1. 集成步骤

1.1 添加依赖

将交付的 AAR 放入 `app/libs/`：

```
dependencies {  
    implementation(files("libs/xmecsdk.aar"))
```

// SDK 运行时传递依赖（AAR 内 `implementation` 声明不自动传递给接入方，须显式引入）

// 版本须与 SDK 一致，以交付文档为准

```
implementation("androidx.media3:media3-exoplayer:1.4.1")  
implementation("androidx.media3:media3-session:1.4.1")  
implementation("androidx.media:media:1.7.0")  
implementation("androidx.security:security-crypto:1.1.0-alpha06")  
implementation("com.squareup.okhttp3:okhttp:4.12.0")  
implementation("androidx.core:core-ktx:1.12.0")  
implementation("com.google.code.gson:gson:2.10.1")  
implementation("androidx.palette:palette-ktx:1.0.0")
```

```
implementation("org.jetbrains.kotlin:kotlinx-coroutines-
android:1.8.0")
implementation("org.jetbrains.kotlin:kotlinx-serialization-
json:1.6.3")
}
```

1.2 权限配置

复制代码

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"
/>
<uses-permission android:name="android.permission.WAKE_LOCK" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission
android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

以下组件已随 AAR 声明（manifest 会自动 merge，无需接入方手动注册）：

复制代码

```
<service android:name="com.xmlly.ecosdk.android.session.PlaybackService"
android:foregroundServiceType="mediaPlayback"
android:exported="false" />
<receiver
android:name="com.xmlly.ecosdk.android.session.MediaButtonReceiver"
android:exported="false">
    <intent-filter><action
android:name="android.intent.action.MEDIA_BUTTON" /></intent-filter>
</receiver>
```

支付 scheme 可见性声明（targetSdk 30+ 必需）：Android 11+ 引入包可见性限制，`startActivity(wechat://)` 等支付拉起在未声明 `<queries>` 时会静默失败（`ActivityNotFoundException`，被 SDK catch 后仅打 debug 日志）。接入方需在 manifest 根节点声明支付 scheme 可见性（scheme 白名单与 SDK `WebViewBridge.handleUrl` 三端对齐）：

复制代码

```
<queries>
    <intent>
        <action android:name="android.intent.action.VIEW" />
```

```

<category android:name="android.intent.category.BROWSABLE" />
<data android:scheme="weixin" />
</intent>
<intent>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="alipays" />
</intent>
<intent>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="alipay" />
</intent>
</queries>

```

1.3 初始化

渠道密钥已内置在交付产物中，初始化无需传参。在 `Application.onCreate()` 调用：

▼
复制代码

```

class App : Application() {
    override fun onCreate() {
        super.onCreate()
        XMEcoSDK.initialize(this)
    }
}

```

日志开关（零配置，自动跟随打包类型）： SDK 日志级别默认自动跟随接入方的打包类型

测试包（`debug` 构建或 `debuggable=true` 的测试 release）输出 DEBUG 全量日志；正式包（非 `debuggable`）静默为 WARN（仅告警/错误，且 URL 中的 `access_token` / `mobile` 等敏感参数自动脱敏为 `***`）。接入方无需任何配置。

如需强制覆盖（联调强制全量 / 最保守强制静默），显式传入：

复制代码

```
XMEcoSDK.initialize(this, debug = true)    // 强制 DEBUG 全量
XMEcoSDK.initialize(this, debug = false)   // 强制 WARN（忽略测试包自动开
启）
```

2. WebView 集成

复制代码

```
class WebViewActivity : AppCompatActivity() {
    private var bindingId: String = ""
    private lateinit var webView: SystemWebView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate()
        webView = SystemWebView(this)
        (webView.view as WebView).apply {
            settings.javaScriptEnabled = true
            settings.domStorageEnabled = true
        }
        bindingId = XMEcoSDK.bridge.setWebView(webView) // 必须在
loadUrl 之前

        XMEcoSDK.setEventListener { event ->                // 监听 H5 事件
            (如关闭页面)
            if (event.module == "UtilBridge" && event.method ==
"closePage") finish()
        }

        (webView.view as WebView).webViewClient = object :
WebViewClient() {
            override fun shouldOverrideUrlLoading(view: WebView?,
request: WebResourceRequest): Boolean {
                // 支付 scheme 拦截 (weixin://、alipays:// 等) 由 SDK 内聚
处理

                if (view == null) return false
                return WebViewBridge.handleUrl(view,
request.url.toString())
            }
            override fun onPageFinished(v: WebView?, url: String?) {

```

```
// 注入 SDK H5 桥接脚本（每次页面导航完成后回调，需每次注入）
AdapterBridge.injectPolyfill(webView)
```

```
}
}
webView.loadUrl(SdkUrls.H5_BASE_URL)
}

override fun onDestroy() {
    super.onDestroy()
    XMEcoSDK.bridge.release(bindingId)
}
}
```

2.1 容器标题栏适配（URL 判断）

SDK 内的 H5 页面（播放器、专辑详情、权益等）自带标题栏；外部页面（支付中间页等）不带标题栏且无 bridge 适配。容器在页面加载时按 URL 判断：URL 含 `H5_BASE_URL` 的为 SDK 页面 → 隐藏原生标题栏（H5 自带）；其余为外部页面 → 显示原生标题栏。纯容器行为，H5 侧零适配。

```
// WebViewClient.onPageStarted 里按 URL 粗判
override fun onPageStarted(view: WebView?, url: String?, favicon:
Bitmap?) {
    val isSdkPage = url?.startsWith(SdkUrls.H5_BASE_URL) == true
    toolbar.visibility = if (isSdkPage) View.GONE else View.VISIBLE
}
```

返回按钮建议：优先 `webView.canGoBack() ? goBack() : finish()`。

跨端框架（Flutter / RN 等）：URL 判断在原生 WebView 导航回调里完成后，通过 Channel 把显隐 boolean 推给 UI 层即可，不需要在跨端层拦截 URL。

3. SDK 接口

3.1 播放控制

通过 `XMEcoSDK.audioService` 访问：

```
复制代码
```

```

val audio = XMecoSDK.audioService

// playContent 完整参数:
albumId/trackId/listenPackageId/breakSecond/enableLocalBreakpointResume
(后 3 参可选)
audio.playContent(
    albumId = "13396678", trackId = "72982155",
    listenPackageId = "0", breakSecond = 0.0,
    enableLocalBreakpointResume = true
)
audio.pause(); audio.resume(); audio.stop(); audio.toggle() // 基本控制
audio.next(); audio.prev() // 切换曲目 (自动处理排序/翻页)
audio.seekTo(timeMs = 30000) // 跳转 (毫秒)
audio.setPlaybackSpeed(rate = 1.5f) // 倍速 0.5-3.0

```

3.2 外部播放委托 (渠道禁用 native 播放时)

渠道 `nativePlaybackAllowed=false` (如 noetixrobotics) 时, SDK 不执行本地播放。H5 页面里用户点播放会被 SDK 拦截, 默认只发错误事件 (接入方收不到播放意图)。

若接入方需要用自己的播放器播放, 注册 `ExternalPlaybackProvider`, SDK 会在拦截时把 H5 传入的播放参数回调给接入方:

 复制代码

```

XMecoSDK.setExternalPlaybackProvider { request ->
    // request: ExternalPlaybackRequest — H5 的播放意图
    // albumId      String    专辑 ID (必填)
    // trackId      String?   曲目 ID (H5 只指定专辑时为 null)
    // trackTitle   String?   曲目标题
    // duration     Long?     曲目时长, 秒
    // breakSecond  Int?      断点续播位置, 秒
    //
    // 在此用自己的播放器播放, 例如:
    myPlayer.play(
        albumId = request.albumId,
        trackId = request.trackId,
        title = request.trackTitle ?: "",

```

```
        seekTo = (request.breakSecond ?: 0) * 1000L,
    )
}
```

说明：

- 回调在 SDK 拦截 native 播放后立即触发（主线程），接入方可在其中启动自己的播放器。
- 封面（cover_url）不在回调参数内——拦截发生在 SDK 拉取专辑详情之前，此时无封面。如需封面，接入方可用 `albumId` 自行查询。
- 未注册时维持原行为（H5 播放被拦截，接入方收不到意图）。

高级播放设置

复制代码

```
// 定时停止（PlayerSleepStyle 枚举，完整 5 参）
// sleepStyle/second/episodes/playStopEpisodes/isOpen
// CUSTOM_SECS 时 second 为秒数，CUSTOM_MINS 时 second 为分钟数，集数样式
// 时 episodes 为集数
audio.playerTimedStop(
    sleepStyle = PlayerSleepStyle.THIRTY_MINS, second = 0, episodes = 0,
    playStopEpisodes = false, isOpen = true
)
// TimerStateInfo:
// sleepStyle/remainingSeconds/remainingEpisodes/playStopEpisodes/isOpen
// (5 字段)
val timer = audio.getPlayerTimedStop()

// 专辑排序/跳头跳尾
audio.setAlbumSort(albumId = "13396678", isAsc = true)
audio.setSkipPlay(albumId = "13396678", skipBeginning = 30, skipEnding = 15)

// 专辑级倍速（useCurrentAlbum=true 仅该专辑，false 全局；apply=true 持久化）
audio.setPlaybackSpeed(rate = 1.5f, albumId = "13396678",
    useCurrentAlbum = true, apply = true)
audio.getPlaybackSpeed(albumId) { info -> /* PlaybackSpeedInfo:
    rate/useCurrentAlbum */ }

// 上下首可用状态
```

```
val trackInfo = audio.hasPreAndNextTrack() // PreAndNextTrackInfo:
hasNextTrack/hasPrevTrack
```

状态查询

复制代码

```
audio.isPlaying() // Boolean
audio.getCurrentPlayable() // PlayableTrack?
(id/title/artist/cover_url/duration/album_id...)
audio.getPlaylist() // List<PlayableTrack>
audio.getSpeed() // Float
```

事件监听

AudioPlayerListener (可选重写需要的方法) :

复制代码

```
audio.addPlayerListener(object : AudioPlayerListener() {
    override fun onPlaybackStarted(track: PlayableTrack?) {}
    override fun onPlaybackPaused(track: PlayableTrack?) {}
    override fun onPlaybackCompleted() {}
    override fun onPlaybackError(msg: String, code: Int, track:
PlayableTrack?) {}
    override fun onPlaybackProgressUpdate(pos: Long, dur: Long, pct:
Double) {} // 秒
    override fun onFloatingPlayerStateChanged(visible: Boolean) {}
// 共 15 个回调, 详见 AudioPlayerListener
})
```

3.2 播控门控

复制代码

```
XMEcoSDK.setNativePlaybackAllowed(false) // 禁用原生播控 (false 时暂停+后
续播控收 2600 错误)
XMEcoSDK.setNativePlaybackAllowed(true) // 恢复 (不会自动 resume)

// 音频焦点主动控制
XMEcoSDK.requestAudioFocus() // 返回 Boolean (是否授予)
XMEcoSDK.abandonAudioFocus()
```

XMEcoSDK.setAudioFocusHeld(true) // true-请求 false-释放

3.3 登录

复制代码

```
// 登录状态查询（异步回调）
XMEcoSDK.checkLoginStatus { status -> /*
isLogin/loginType/hasValidToken/hasUserProfile/accessToken */ }
XMEcoSDK.isUserLogin { isLogin -> }
XMEcoSDK.isVipUser { isVip -> }
XMEcoSDK.getUserNickname { name -> }; XMEcoSDK.getUserAvatarUrl { url -
> }; XMEcoSDK.getUserId { uid -> }

// 登录（两种方式，按渠道接入约定选用）
// ① 票据登录: ticket 直接换取 token
XMEcoSDK.loginWithThirdPartyTicket(ticket) { result -> /* LoginResult:
success/loginType/message */ }
// ② 三方 token 登录: SDK 内部将 ticket 拼成 {"ticket":"..."} 作为
third_info 透传
XMEcoSDK.loginWithThirdPartyTokenByTicket(ticket) { result -> /*
LoginResult: success/loginType/message */ }

// 自动确保登录（未登录时取 deviceInfoProvider 返回的三方信息 JSON 透传登录）
// deviceInfoProvider 返回 JSON 字符串，字段同
VipTicketProvider.getThirdDeviceInfo (device_id/ticket/model)
XMEcoSDK.ensureLoginWithThirdPartyAuth(
    deviceInfoProvider = {
        JSONObject().apply {
            put("device_id", ThirdParty.deviceId)
            put("ticket", ThirdParty.ticket ?: "")
        }.toString()
    },
    callback = { result -> /* LoginResult: success/loginType/message */
}
)

// ③ 登录并按权益状态决定是否建立登录态（耦合登录与权益领取）
// 适用场景：接入方希望在登录时校验是否有生态权益，无权益则不建立登录态（避免无
效登录占用配额）
XMEcoSDK.loginAndClaimIfNeeded(
```

```

loginInfoJson = ""{"ticket":"xxx"}"", // 登录凭据 JSON，同
loginWithThirdPartyToken 的 third_info
// ticket（此处ticket是手
机号加密后的登录票据）
vipDeviceInfo = null, // 权益查询设备信息 JSON，
为空时回退 VipTicketProvider.getVipDeviceInfo()
// 含 device_id + ticket
（此处ticket是设备SN加密后的票据）
) { result ->
    // result: LoginAndClaimResult
    // result.loginResult.success -> 是否已建立/保持登录态
    // result.benefitQuery?.status -> 权益状态（0..4，见
EcoBenefitStatus），null=未查到/查询失败
    // result.benefitQuery?.claimed -> 是否触发了领取（仅 status==4）
    // result.benefitQuery?.claimSuccess -> 领取结果（claimed=true 时有意义）
}

// 监听器
XMEcoSDK.addLoginStatusListener(object : ExternalLoginListener {
    override fun onLoginSuccess(status: LoginStatus) {}
    override fun onLoginFailed(error: String, code: ExternalLoginError)
{}
    override fun onLogout() {}
})

// 登录成功数据透传（实际登录成功时触发，影子登录除外）
// 覆盖：SDK 主动登录（loginWithThirdPartyTicket/loginAndClaimIfNeeded 转
正）+ H5 同步登录（updateAuthEntity）
// 影子登录（shadowLogin，仅查权益的临时 token，不落库）不触发
XMEcoSDK.setLoginDataListener { json ->
    // json:
{"access_token":"...", "refresh_token":"...", "uid":3192, "login_type":"THI
RD_PARTY"}
    // login_type: TICKET / MOBILE / THIRD_PARTY（区分登录来源，便于埋
点）

```

ExternalLoginError: INVALIDTICKET(1001) / INVALIDMOBILE(1002) / LOGINFAILED(1003)
/ NETWORKERROR(1004) / TOKENREFRESHFAILED(1005) / TOKEN_EXPIRED(1006)

3.4 权益注入

注册 VIP 票据提供者，SDK 在权益查询/领取时委托回调。

各方法说明：

方法	必填	返回类型	说明
<code>getVipTicket</code>	✓	<code>String</code>	VIP 票据（JWT 字符串），未注入时 H5 调用会抛 <code>IllegalStateException</code>
<code>getVipDeviceInfo</code>	○	<code>String?</code>	本机设备信息 JSON 字符串 ，默认 <code>"{}"</code> ；返回 <code>null</code> 由 SDK 兜底为 <code>"{}"</code>
<code>getThirdDeviceInfo</code>	○	<code>String?</code>	三方设备信息 JSON 字符串 ，默认 <code>"{}"</code> ；返回 <code>null</code> 由 SDK 兜底为 <code>"{}"</code>
<code>onVipClaimResult</code>	✓	<code>Unit</code>	权益领取结果回调， <code>result</code> 为结果 JSON 字符串 。 <code>fetchEcoBenefitQualification(autoClaim=true)</code> 自动领取的结果也走此回调（ <code>{"success":bool,"code":int,"msg":"..."}</code> ）

⚠ `getVipDeviceInfo` / `getThirdDeviceInfo` 返回的是 **JSON 字符串**（如 `"{\\"device_id\\":\\"xxx\\"}"`），不是 `JsonObject` / `Map`。用 `JsonObject().toString()` 构造。

JSON 字段格式（接入方按渠道需要填充，无需的字段可不填）：

方法	字段	类型	示例	说明
----	----	----	----	----

// 【可选】三方设备信息（此处ticket是手机号加密），返回 JSON 字符串；无三方信息返回 null（SDK 兜底 "{}"）

```
override suspend fun getThirdDeviceInfo(): String? {  
    val ticket = ThirdParty.ticket ?: return null  
    return JSONObject().apply {  
        put("device_id", ThirdParty.deviceId)  
        put("ticket", ticket)  
        put("model", android.os.Build.MODEL)  
    }.toString()  
}
```

// 【必填】权益领取结果回调。result 格式：{"success":true,"msg":"领取成功"}

```
override fun onVipClaimResult(result: String) {  
    try {  
        val json = JSONObject(result)  
        Log.i("Vip", "领取结果：  
success=${json.optBoolean("success")}, msg=${json.optString("msg")}")  
    } catch (e: Exception) {  
        Log.e("Vip", "解析领取结果失败：$result", e)  
    }  
}
```

```
private fun isExpired(jwt: String): Boolean = /* 解析 JWT exp 判断  
*/ false  
}
```

// 注册（必须在 initialize 之后、进入权益 H5 页之前；通常放

Application.onCreate）

```
XMEcoSDK.setVipTicketProvider(MyVipTicketProvider())
```

💡 时序：必须 XMEcoSDK.initialize() 之后再注入；建议在 Application.onCreate 或进入权益页的 Activity.onCreate 注入，不要等用户点击才注入。

3.5 数据获取

通过 XMEcoSDK.sdk?.businessService （suspend，在协程内调用）：

```
val biz = XMEcoSDK.sdk?.businessService
```

复制代码

```

biz?.fetchCommonSubscribeList(page = 1, limit = 10) //
List<AlbumInfo>
biz?.subscribeAlbum(albumId); biz?.unsubscribeAlbum(albumId)
biz?.fetchCommonHistoryList(page = 1, limit = 10) //
List<HistoryInfo>
biz?.fetchListenList(page = 1, limit = 10, ids = "1,2") // Map<String,
ListenGroup> (ids 逗号分隔)
biz?.fetchAlbumBatch(ids = "1,2,3") //
List<AlbumInfo> (ids 逗号分隔)
biz?.textAiSearch(text = "三国", limit = "10") //
TextSearchResponse (limit 为 String 类型)
biz?.fetchEcoBenefitQualification(thirdDeviceInfo = null, autoClaim =
false) // EcoBenefitData? (autoClaim=true 时查询到可领权益自动领取, 结果回
调 onVipClaimResult)
biz?.fetchDeviceVipInfo() //
DeviceVipInfo? (subscribe_activity 为权益领取页 URL)
// 权益判断: data?.ecological_benefit_status == 4
(EcoBenefitStatus.HAS_BENEFIT) 表示有权益可领取
biz?.isSubscribeBatch(albumIds = "1,2,3") // Map<String,
Boolean> (albumIds 逗号分隔)
biz?.verifyAlbumPermission(albumId) // Boolean

```

搜索接口 (searchAlbum) 属于 `ContentService` : `XMEcoSDK.sdk?.contentService?.searchAlbum(keyword = "三国", page = 1, pageSize = 20)`, 返回 `ContentPage<TrackItem>`。

`fetchEcoBenefitQualification` 参数说明:

参数	类型	默认值	说明
<code>thirdDeviceInfo</code>	<code>String?</code>	<code>null</code>	三方设备信息 JSON 字符串 (如 <code>{"device_id": "xx"}</code>)。为空时 SDK 自动从 <code>VipTicketProvider</code> 获取

autoClaim	Boolean	false	true 时查询到可领权益 (status==4) 自动领取, 结果回调 onVipClaimResult ; false 时仅查询
-----------	---------	-------	--

3.5.1 登录并领取权益 (loginAndClaimIfNeeded)

loginAndClaimIfNeeded 把"登录"和"权益领取"耦合为原子操作, 按当前登录态分两条路径:

未登录路径 (核心场景):

1. 影子登录: 拿到 token + uid, 不落库、不通知监听器、不切状态机 (临时态, 仅用于查权益)
2. 用影子 token 查询权益资格:
 - 查询技术失败 (网络/解析/querier 未注入) → 不登录, 返回失败
 - status==4 (有权益) → 正式登录 + 自动领取
 - status==0 (无权益) → 不建立登录态, 返回失败
 - 其他状态 (1/2/3) → 正式登录

已登录路径 (用户已登录后再调用):

1. 跳过影子登录, 用现有 token 查询权益资格 (不覆盖当前登录态), 每次调用都回调 setLoginDataListener
2. status==4 (有权益) → 自动领取, 保持登录态
3. 任何其他状态 (无权益/其他/查询失败) → 直接放行 (不退登)

⚠ 并发安全: loginAndClaimIfNeeded 内部用 Mutex 串行化, 并发调用会排队执行。但 SDK 为单全局登录态 (一份 token/uid), 多设备连续调用时第二次走已登录路径用的是第一次设备的 token, 身份与设备可能不匹配。多设备切换建议先 logout() 再调本方法。

💡 领取结果双通道: 领取结果既回调 VipTicketProvider.onVipClaimResult (与 fetchEcoBenefitQualification 一致, {"success":bool,"code":int,"msg":"..."}), 也通过返回值 LoginAndClaimResult.benefitQuery.claimSuccess 提供。接入方可选任一通道处理。

返回字段	类型	说明
loginResult.success	Boolean	是否已建立/保持登录态

loginResult.message	String	"already logged in" = 已登录路径；"无生态权益，未建立登录态" = 未登录无权益
benefitQuery	EcoBenefitQueryResult?	null = 未进入查询（影子登录失败）/ 查询技术失败
benefitQuery.status	Int	权益状态 0..4 (EcoBenefitStatus)
benefitQuery.claimed	Boolean	是否触发了领取（仅 status==4 时为 true）
benefitQuery.claimSuccess	Boolean?	领取结果（claimed=true 时有意义）；未领取时为 null

参数说明：

参数	类型	说明
loginInfoJson	String	登录凭据 JSON，透传为登录接口的 third_info。含 ticket（手机号加密后的登录票据），如 {"ticket": "eyJ0eXA..."}
vipDeviceInfo	String?	权益查询设备信息 JSON，为空时回退 VipTicketProvider.getVipDeviceInfo()。含 device_id + ticket（设备 SN 加密后的 VIP 票据），如 {"device_id": "xxx", "ticket": "eyJ0eXA..."}

⚠ 两个 ticket 含义不同：loginInfoJson 的 ticket 是手机号加密的登录票据（用户维度）；vipDeviceInfo 的 ticket 是设备 SN 加密的 VIP 票据（设备维度，与 getVipTicket() 同源）。

3.6 悬浮播放器

✓ 复制代码

```
// 注入封面点击跳转（启动 WebViewActivity 前设置一次）
FloatingPlayerManager.h5PlayerNavigator = { ctx, url ->
    ctx.startActivity(Intent(ctx,
WebViewActivity::class.java).putExtra("URL", url))
}

XMEcoSDK.showFloatingPlayer(activity)    // 显示（跟随 Activity 生命周期）
XMEcoSDK.hideFloatingPlayer()
XMEcoSDK.isFloatingPlayerVisible()
XMEcoSDK.setDisallowedFloatingPlayerActivities(setOf("com.example.LoginActivity"))

// 点击监听（type: 0=关闭, 1=播控, 2=封面）
XMEcoSDK.setOnMiniPlayerViewClickListener { type, playable, isPlaying -> }
```

3.7 页面跳转监听

H5 调 `DeviceBridge.openPlayer` / `openAlbumDetail` 时回调接入方：

✓ 复制代码

```
XMEcoSDK.addCommonActionListener { action, albumId, trackId ->
    // action: "openPlayer" | "openAlbumDetail"
}
```

3.8 埋点回传

每条音频播放结束（切换/暂停/完成/列表完成）时回调本次累计播放时长：

✓ 复制代码

```
XMEcoSDK.addAnalyticsListener { event ->
    // event.albumId / trackId / playSeconds（本次净播放秒数，已扣除暂停） / startAt（毫秒时间戳）
}
```

3.9 支付

复制代码

```
XMEcoSDK.setPaymentType(1) // 0=SDK支付, 1=接入方支付
XMEcoSDK.getPaymentType() // Int
XMEcoSDK.addHostPaymentListener { orderId, amount, productId, params ->
/* 拉起支付 */ }
XMEcoSDK.addPaymentListener { result -> /* HostPaymentResult:
code/message */ }
XMEcoSDK.notifyHostPaymentResult(HostPaymentResult(code = 0, message =
"成功"))
```

3.10 渠道信息

复制代码

```
XMEcoSDK.getChannelName() // 渠道名称
XMEcoSDK.getAppKey(); XMEcoSDK.getSN(); XMEcoSDK.getChannelId()
XMEcoSDK.isPlayerEnabled() // 播放器是否启用
XMEcoSDK.isEcoLinkModuleAvailable() // EcoLink 模块是否编入
XMEcoSDK.isAIModuleAvailable(); XMEcoSDK.isAdModuleAvailable()
```

3.11 URL 构建

复制代码

```
XMEcoSDK.buildDefaultPlayerUrl(albumId, trackId, breakSecond) // H5 播放
页 URL
XMEcoSDK.buildAlbumDetailUrl(albumId) // H5 专辑
详情 URL
```

4. 错误码

码	说明	处理建议
1000	未知错误	检查参数和网络
1001	网络错误	检查网络连接
2000	播放错误	检查音频资源有效性

2019	音频列表为空	该专辑暂无可播放内容
2026	播放鉴权错误	检查用户权限/登录状态
2501	非版权内容	该内容暂时无法播放
2600	原生播控被禁用	调 <code>setNativePlaybackAllowed(true)</code> 后重试
3000	登录失败	检查登录凭据
3001	令牌已过期	重新登录
3006	用户未登录	引导用户登录

5. 注意事项

- 混淆：**AAR 已内置 `consumer-rules.pro`（随 `consumerProguardFiles` 自动应用），接入方开启 `minifyEnabled` /R8 时 SDK 类、`@Serializable` 序列化器与 `@JavascriptInterface` 方法自动保留，无需手动添加 **keep** 规则。
- 版本号：**SDK 版本 `2.0.0` 不可修改，H5 首页做版本校验。
- 初始化：**必须在 `Application.onCreate` 调用，幂等。
- WebView：**`setWebView` 必须在 `loadUrl` 之前；`onPageFinished` 注入 `polyfill`。
- 权益：**`setVipTicketProvider` 的 `getVipTicket` / `onVipClaimResult` 必填；`getVipDeviceInfo` / `getThirdDeviceInfo` 可选（返回 JSON 字符串，默认 `"{}"`，返回 null 由 SDK 兜底为 `"{}"`）。
- SDK 生命周期与登录态管理（重要）：**
 - 退出登录**只需调 `XMEcoSDK.logout {}`（或 H5 调 `AuthBridge.logout`），清除 token + 通知监听器。
 - 关闭 H5 页面**只需 `onDestroy` 里 `XMEcoSDK.bridge.release(bindingId)` 释放 WebView 绑定。
 - `VipTicketProvider` 的 `getVipTicket` / `getVipDeviceInfo` / `getThirdDeviceInfo` 是 `suspend` 方法，必须在合理时间内返回。如果内部有异步网络请求，务必设超时兜底，避免永久挂起导致 H5 桥超时。